Lab 03b
AI-Assisted Software Engineering

Gianluca Aguzzi[1] Mirko Viroli[1]

[1]Alma Mater Studiorum – Università di Bologna
{gianluca.aguzzi, mirko.viroli}@unibo.it

March 13, 2026

One slide sum-up on AI-Assisted Software Engineering

AI Coding Agents

- Tools like GitHub Copilot embed LLMs directly in the IDE
- They can generate, complete, refactor, test, and explain code
- Output quality depends heavily on **context** and **prompt design**
- Human-in-the-loop remains essential: the developer guides, reviews, and owns the code

Key dimensions to explore

- **Prompt Engineering** – explicit instructions shaping the agent's behaviour (`copilot-instructions.md`)
- **Context Engineering** – enriching the project with documentation the agent can leverage
- **Agent specialisation** – focused agents for testing, refactoring, debugging, and more
- **Vibe coding vs. AI-assisted development** – letting AI drive vs. owning the codebase yourself

Starting point and goals

References

- GitHub Copilot (VS Code): `https://marketplace.visualstudio.com/items?itemName=GitHub.copilot`
- GitHub Copilot (JetBrains): `https://plugins.jetbrains.com/plugin/17718-github-copilot`
- GitHub Education benefits: `https://github.com/settings/education/benefits`
- 03b-repo: `https://github.com/cric96/asmd25-03b-code-ai-assisted-soft-dev`
- Slide 03b provides an overview of AI-assisted engineering concepts

General goals

- set up and explore GitHub Copilot in your IDE
- experiment with prompt and context engineering to improve code quality
- practise AI-assisted development with a human-in-the-loop approach

Operational Steps: AI-Assisted Development (1/2)

Step 0 – Set Up Copilot

- Install the GitHub Copilot extension for your IDE
  - VS Code: `https://marketplace.visualstudio.com/items?itemName=GitHub.copilot`
  - JetBrains: `https://plugins.jetbrains.com/plugin/17718-github-copilot`
- Activate GitHub Education benefits at `https://github.com/settings/education/benefits`
- Download and set up the project from `https://github.com/asmd-2025/lab-03b-ai-assisted`

Step 1 – Baseline: "Try the Vibe"

- Ask Copilot to implement a **Connect Four** game with *minimal* guidance
  - The interface `it.unibo.ConnectFour` is provided as a starting point
- Evaluate the solution along three axes:
  - **Code quality** – Are there code smells? Is the code readable and maintainable?
  - **Correctness** – Does the game work as expected? Are edge cases handled?
  - **Creativity** – Are interesting techniques used? Is the domain well understood?
- Try multiple Copilot models (if available), save each solution, and compare
- Also try open-source models via Ollama or other providers and compare

Step 2 – Prompt Engineering

- Create `.github/copilot-instructions.md` with explicit coding guidelines and constraints
- Regenerate the Connect Four game and evaluate using the same axes as Step 1
  - Is the output more consistent with your instructions?
  - How much did the instructions improve (or change) the solution?

Operational Steps: AI-Assisted Development (2/2)

Step 3 – Context Engineering

- Enrich the project with supporting documentation, for example:
  - `PRODUCT.md` – product vision, features, and requirements
  - `CONTRIBUTING.md` – coding standards, conventions, and guidelines
- Update `.github/copilot-instructions.md` to reference this context; regenerate and compare

Step 4 – Human-in-the-Loop: AI-Assisted Development

- Put yourself in the loop alongside Copilot; choose one or more strategies:
  - **Test after generation** – write tests to verify correctness of generated code
  - **TDD** – write tests first, then let Copilot generate the implementation
  - **Iterative refactoring** – improve quality through guided, incremental changes
- Create specialised *agents* for specific tasks (e.g., a *test agent*, a *refactor agent*)
- You are the **owner** of the codebase – guide Copilot as a tool, not a replacement

Tasks R&D

REENGINEER

Goal: Apply AI-assisted engineering to one of your existing projects.
Primary Task: Use Copilot (with appropriate prompt and context engineering) to refactor a module of your project. Reflect on the quality of the result and the effort needed to guide the agent.
Advanced Task: Starting from a `README.md`, use a requirements agent to structure and formalise the requirements, then implement the system with Copilot assistance. As a concrete example, try applying this to the `/src/main/resources/product.md` exercise.

AGENT-SPECIALISATION

Goal: Design a multi-agent workflow where specialised Copilot agents collaborate to produce a higher-quality software artefact.
Primary Task: Build at least two specialised agents (e.g., a design agent, a test agent, a refactor agent) and evaluate whether their combination outperforms a single generic agent.
Please, look at `https://code.visualstudio.com/docs/copilot/overview` to more details on how to create specialised agents with Copilot.

EVALUATION

Goal: Systematically measure how prompt and context engineering affect the quality of AI-generated code.
Primary Task: Define a small benchmark (a few programming tasks) and compare results across: (1) no instructions, (2) `copilot-instructions.md` only, and (3) full context engineering.
Advanced Task: Investigate the effect of different Copilot models or providers on the same benchmark.